SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 10)

TASK 10 – MEASURE AND COMPARE

The purpose of this task is to test the earlier design judgements on a Linux machine. Timings vary with CPU frequency, memory pressure, kernel, compiler, filesystem, storage device and system load. Record the Linux distribution, kernel version, compiler version and exact commands with every result.

10.1 Measure copy-program system calls

```
# Compiler and OS details
gcc --version | head -1
uname -a
cat /etc/os-release

# Program output
./task06_copy

# System-call summary
strace -c -e trace=read,write,openat,close ./task06_copy

# Detailed read trace
strace -e trace=read -o task06-read.trace ./task06_copy
```

Compare the program's own read() count with strace. The program counter counts only read() calls inside copy_file(). strace may also show startup and library reads. Filter or inspect the detailed trace so that the application loop is distinguished from startup activity.

10.2 Measure user and kernel time

```
/usr/bin/time -v ./task06_copy
# or, where the shell time command is used:
time ./task06_copy
```

The time utility reports elapsed wall-clock time and may report user CPU time and system CPU time. User time measures CPU execution in user mode; system time measures CPU time spent in kernel mode on behalf of the process. Wall-clock time also includes waiting and scheduling delays, so it is not necessarily equal to user plus system time.

10.3 C program to compare 1,000 process and thread launches

The following program performs 1,000 fork()+waitpid() cycles and 1,000 pthread_create()+pthread_join() cycles. It measures creation plus completion for each method. The operations are performed sequentially to avoid requiring 1,000 simultaneous tasks and to reduce the chance of exhausting process or thread limits. Use the same machine and repeat the run several times

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#define COUNT 1000

static double seconds_between(struct timespec a, struct timespec b) {
    return (double)(b.tv_sec - a.tv_sec) +
           (double)(b.tv_nsec - a.tv_nsec) / 1000000000.0;
}

static void *thread_job(void *arg) {
    (void)arg;
    return NULL;
}

int main(void) {
    struct timespec start, end;

    if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
        perror("clock_gettime"); return 1;
    }
```

```
for (int i = 0; i < COUNT; i++) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(0);
    int status;
    while (waitpid(pid, &status, 0) < 0) {
        if (errno == EINTR) continue;
        perror("waitpid"); return 1;
    }
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        fprintf(stderr, "child failed at iteration %d\n", i);
        return 1;
    }
}
if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
    perror("clock_gettime"); return 1;
}
printf("1000 fork()+waitpid() cycles: %.6f seconds\n",
       seconds_between(start, end));

pthread_attr_t attr;
if (pthread_attr_init(&attr) != 0) {
    fprintf(stderr, "pthread_attr_init failed\n"); return 1;
}
/* A smaller stack makes this benchmark practical on many systems. */
int rc = pthread_attr_setstacksize(&attr, 256 * 1024);
if (rc != 0) {
    fprintf(stderr, "pthread_attr_setstacksize: %d\n", rc);
    pthread_attr_destroy(&attr); return 1;
}

if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
    perror("clock_gettime"); pthread_attr_destroy(&attr); return 1;
}
```

```
    for (int i = 0; i < COUNT; i++) {
        pthread_t thread;
        rc = pthread_create(&thread, &attr, thread_job, NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_create at %d: %d\n", i, rc);
            pthread_attr_destroy(&attr); return 1;
        }
        rc = pthread_join(thread, NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join at %d: %d\n", i, rc);
            pthread_attr_destroy(&attr); return 1;
        }
    }
    if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
        perror("clock_gettime"); pthread_attr_destroy(&attr); return 1;
    }
    printf("1000 pthread_create()+join() cycles: %.6f seconds\n",
           seconds_between(start, end));
    pthread_attr_destroy(&attr);
    return 0;
}
```

```
gcc -std=c11 -Wall -Wextra -O2 task10_bench.c -pthread -o task10_bench
./task10_bench
/usr/bin/time -v ./task10_bench
```

10.4 Measurement table – fill with your actual Linux results

Measurement	Run 1	Run 2	Run 3	Average / observation
read() calls, 1-byte buffer				Expected program count: 10,001
read() calls, 512-byte buffer				Expected program count: 21

Measurement	Run 1	Run 2	Run 3	Average / observation
read() calls, 4,096-byte buffer				Expected program count: 4
Copy program user CPU time				Record from time output
Copy program system CPU time				Record from time output
Copy program elapsed time				Record from time output
1,000 fork()+waitpid() cycles				Record from task10_bench
1,000 thread create+join cycles				Record from task10_bench

Required before submission: Do not type estimated or invented times into this table. Run the programs on your Linux machine, paste the terminal output or screenshots into the matching GitHub task folder, and then enter the measured values.

10.5 How to interpret the measurements

- The 4,096-byte buffer should make far fewer read() calls than a one-byte buffer. If elapsed time does not improve proportionally, the file may be cached, storage latency may be low, or other overhead may dominate.
- System time is expected to be more sensitive to syscall count because each read()/write() enters the kernel. User time includes the program's copy loop and memory operations.
- Thread creation is generally lighter than process creation because threads share the same process address space. However, measured values depend on the C library, kernel, stack size and synchronization behaviour.
- The benchmark above measures launch plus termination and join/wait overhead, not a pure isolated creation instruction. State this limitation when interpreting the result.

- Run each test several times under low system load. Do not compare results from different machines as if they were controlled measurements.

10.6 Alternative: threads instead of processes

Threads normally require less memory and setup because they share code, heap and global variables within one process. Each thread still needs its own stack, register state, thread-local storage and kernel scheduling structures. Processes have separate virtual address spaces; fork() uses copy-on-write to share unchanged pages initially, but each process still needs its own process structures and page-table management. For a workload where isolation and independent failure are important, processes are safer. For tightly coordinated parallel work inside one application, threads can be more efficient if synchronization is correct.